
TENSIXRT-VLA: CO-DESIGNING FLOW-MATCHING VLA INFERENCE FOR TILED MANY-CORE ACCELERATORS

Wonchan Son¹

ABSTRACT

Vision–language–action (VLA) policies increasingly produce robot actions by iterating a flow-matching action expert over a cached perception context, placing an unusual workload inside a closed control loop: a large prefill executed once per decision, followed by K repeated executions of a weight-dominated expert, optionally across C candidate trajectories that share both weights and context. We present TensixRT-VLA, a case study that maps the complete post-VLM action-generation path of SmolVLA onto a Tensix-oriented dataflow backend and evaluates it in *TT-Lang functional simulation*; *no physical Tenstorrent silicon is measured in this work*. On the canonical free-running $K=10$ reference case the backend matches PyTorch within 8.3×10^{-7} maximum absolute error; across a 240-flow stress test the worst final-action error is 3.4×10^{-5} . Exploiting workload invariants yields structural results: a streaming SwiGLU that attains the derived padded tile-traffic lower bound under the stated tiling and DRAM-residency assumptions (10,990→4,508 simulator-observed tiles, 59% fewer transfers, eliminating intermediate DRAM materialization); hoisting of flow-invariant prefix key/value projections ($C \cdot K \cdot 16 \rightarrow 16$ calls, bitwise-equivalent); and candidate-parallel mappings in which, in the fused-MLP candidate experiment, batching amortizes weight tiles as $1/C$ over tested $C \in \{1, 2, 4, 8\}$ and a two-node multicast halves weight DRAM reads with bitwise-identical outputs. Closed-loop LIBERO experiments expose a cautionary result: single-decision counterfactual evaluation judges reduced flow depth nearly free, yet sustained low-depth execution is significantly worse (paired exact McNemar $p=0.0075$, $n=125$) while $K=6$ shows no detected difference from $K=10$; intervention rollouts suggest that much of the deficit is induced by early shallow decisions and is not visibly recovered by later full-depth execution—in this setting, single-decision counterfactual labels understate the closed-loop performance penalty of reduced flow depth.

1 INTRODUCTION

Vision–language–action policies couple an expensive perception backbone to an iterative action generator and place both inside a robot control loop: every action chunk requires a fresh inference pass, and the quality of that pass feeds back into the states the policy will face next. Inference cost is therefore not an offline serving metric but a participant in closed-loop dynamics (Shukor et al., 2025; Black et al., 2025a).

From a systems perspective, flow-matching VLA inference is not a generic one-pass Transformer workload. In SmolVLA (Shukor et al., 2025), a perception prefix is computed once per decision and then held fixed while a 16-layer action expert executes $K=10$ times over only 50 action tokens; candidate action trajectories, when sampled, differ only in their initial noise and share every weight and the entire perception context. Repetition, invariance, and sharing

dominate the execution structure—exactly the properties that tiled many-core accelerators with explicit dataflow expose directly and that generic operator libraries do not.

Existing acceleration work largely targets single-pass model inference or generic operators; efficient-VLA efforts operate at the model or serving level—distillation to fewer denoising steps, quantization, token caching and pruning, early exit (Prasad et al., 2024; Wang et al., 2025a; Xu et al., 2025; Yue et al., 2024)—and operator-fusion work on tiled accelerators targets generic LLM decoding (Wu et al., 2026). *What these leave open, to our knowledge, is a cross-layer account that connects the iterative structure of flow VLA inference to a concrete many-core dataflow mapping and then to closed-loop robot behavior (§7).*

TensixRT-VLA provides that account for SmolVLA on the Tensix architecture using the TT-Lang programming model. The VLM prefill remains in PyTorch; the complete post-VLM floating-point path—suffix embedding, timestep conditioning, all 16 expert layers (RMSNorm, grouped-query attention with RoPE, fused SwiGLU), final projection, and Euler integration—executes as TT-Lang kernels in func-

¹Independent Researcher, Seoul, Republic of Korea. Correspondence to: Wonchan Son <wonchan.son99@gmail.com>.

tional simulation, with no silent fallback.

Our evaluation yields four findings. First, the backend preserves model behavior through complete free-running integration (8.3×10^{-7} final-action error at $K=10$ ^[PT]), with per-layer divergence flat across depth. Second, streaming the SwiGLU intermediate reaches the derived padded tile-traffic lower bound under the stated assumptions^[SIM]. Third, invariant and shared work amortizes strongly: prefix K/V projection calls drop $C \cdot K \cdot 16 \rightarrow 16$ (bitwise-equivalent), batched candidates consume weight tiles at exactly $1/C$ in the fused-MLP mapping (tested $C \in \{1, 2, 4, 8\}$), and a two-node multicast halves weight DRAM reads^[SIM]. Fourth, and unexpectedly, the robot-side analysis inverts a natural assumption: counterfactual single-decision evaluation, performed with bit-exact simulator branching, finds reduced flow depth almost always sufficient, while sustained reduced depth costs 9.6 success points in closed loop (0.320 vs. 0.416, paired $p=0.0075$)^[ROB]—a bias we analyze as the K *paradox*.

Our contributions are:

- **Workload characterization.** We measure SmolVLA’s post-VLM action generation and identify invariant prefix context, iterative weight-dominated expert execution (~ 99.8 M parameters touched per step vs. 144 KB of resident hidden-state activation^[AN]), candidate-level sharing, and tile geometry that shapes any tiled mapping (§3).
- **Complete post-VLM backend.** We implement the entire SmolVLA action-generation path as Tensix-oriented TT-Lang kernels and validate it layer-by-layer, per denoise step, and through free-running $K=6/K=10$ integration against PyTorch on real captured tensors (§4, §6.1).
- **Dataflow and reuse transformations.** We design and measure streaming SwiGLU, GQA K/V reuse, prefix-K/V projection hoisting, candidate batching, and multi-node weight multicast, reporting simulator-observed structural metrics and model-level arithmetic only (§6.2, §6.3).
- **Accelerator-robot connection.** We measure the fixed- K compute-quality frontier on LIBERO and find that, in our evaluated setting, single-decision counterfactual labels can substantially underestimate the cost of reduced flow depth, motivating sequential rather than greedy compute allocation (§6.4).

2 BACKGROUND

SmolVLA action generation. SmolVLA (Shukor et al., 2025) couples a frozen-vision SmolVLM2 backbone (16 text layers used, hidden 960) to a width-reduced action

Table 1. SmolVLA post-VLM workload per decision (hooked-shape measurement^[PT]; MACs analytical^[AN]).

Prefix tokens (2 cameras + language + state)	177
Action suffix tokens (H)	50
Expert layers / hidden / MLP	16 / 720 / 2048
GQA (query / KV heads $\times$ dim)	15 / 5 $\times$ 64
Flow steps K (default)	10
Prefill (linear ops, incl. vision)	203.3 GMAC
Reference expert step (w/ prefix reproj.)	5.37 GMAC
Optimized repeated step (post-hoist)	5.08 GMAC
SwiGLU share (ref. / opt.)	66% / 70%
Parameters touched per step	99.8 M
Action hidden state [50, 720], fp32	144 KB
MLP intermediate [50, 2048], streamed	409.6 KB

expert (hidden 720, 16 layers) via interleaved attention: even layers perform joint self-attention with the perception tokens, odd layers cross-attend to the cached perception key/value tensors. A decision consumes a 177-token prefix (two camera views, language, state) and produces an $H=50$ -step action chunk.

Flow matching. Actions are generated by integrating a learned velocity field from noise: with $x_0 \sim \mathcal{N}(0, I)$ (so $t_0=1$), the implementation iterates, for $k = 0, \dots, K-1$ with $\delta = -1/K$ and $t_k = 1 + k\delta$,

$$x_{k+1} = x_k + \delta v_\theta(x_k, t_k, \text{ctx}), \quad (1)$$

where v_θ is one full 16-layer expert forward over the 50 action tokens and ctx is the fixed prefix context. Default $K=10$. C denotes candidate trajectories drawn from distinct x_0 ; all candidates share θ and ctx. The Euler step size $\delta = -1/K$ changes with K , so the integration *trajectories* of different depth are not nested: even where evaluation times t_k coincide, a $K=2$ run’s intermediate states do not match those of a $K=6$ or $K=10$ run, so a shallow trajectory cannot be continued into a deeper one with equivalent semantics—depth must be committed before integration.

Tensix and TT-Lang. Tensix-class devices are grids of cores with ~ 1.4 MB local SRAM (L1) each, explicit DRAM/NoC data movement, and 32×32 -tile compute primitives. TT-Lang exposes this model as Python-authored kernels with dataflow buffers (DFBs), explicit copies, and inter-node pipes; its functional simulator executes kernels hardware-free and reports structural metrics: per-tensor DRAM/L1 tile transfers, DFB events, and L1 capacity accounting. These metrics characterize the *authored dataflow*, not silicon timing (§8).

3 WORKLOAD CHARACTERIZATION

Table 1 summarizes the measured structure of one decision.

Four observations shape the design. **(O1) The expert**

is **weight-dominated**. Each of the K steps re-touches $\sim 99.8\text{M}$ parameters while activations stay within a single core’s L1; weight movement, not arithmetic, dominates the structural cost profile. The arithmetic intensity makes this concrete: the repeated (post-hoist) expert step is ~ 10.2 GFLOP against a weight working set of 200 MB in bf16, i.e. ≈ 51 FLOP/byte^[AN]. Because only the H action tokens pass through weights re-read every step, this intensity is pinned near H regardless of hidden width (Williams et al., 2009). At ≈ 51 FLOP/byte the operational intensity is low enough that repeated weight movement is a primary *structural* concern; whether it becomes the realized bottleneck depends on the device and requires on-silicon measurement (§8). A natural response—keeping the full bf16 expert resident in the assumed single-Wormhole on-chip SRAM/L1 budget—is infeasible: the bf16 expert (~ 200 MB) exceeds that aggregate budget (~ 105 MB: $72 \text{ cores} \times \sim 1.46 \text{ MB per our capacity derivation, appendix}$; the exact figure is device- and reservation-dependent), and spatial per-layer pinning falls short by $\sim 2\times$ per core group^[AN]. This negative result redirects weight amortization to axes that batch *independent work against one weight pass*. **(O2) The prefix context is invariant** across all K steps and all C candidates. Beyond the standard reuse of cached perception K/V, we find the implementation re-applies the expert’s own K/V projections to that constant cache inside every step; since only queries receive rotary embedding on the cross-attention path, the projected result is step-invariant—repeated work that can be hoisted outside the loop entirely. **(O3) Candidates multiply reuse**. Candidate trajectories differ only in x_0 : prefix, projections, and weights are shared, so the marginal candidate is algorithmically cheap (total work at $C=8$ is $2.5\times$ that of $C=1$ at $K=10$, not $8\times$ ^[AN]). **(O4) Tile geometry is non-trivial but benign**. Hidden 720 pads to 23 tiles (+2.2% on weights); the 50-token suffix pads to 64 rows (+28% on activation matmuls), an overhead that candidate batching absorbs (+4% at $C=8$). The 23-tile contraction is prime; we evaluated re-padding to 24 tiles and found it strictly worse under our streaming schedule (+4.2% tiles, $2\times$ DFB events^[SIM]), because the schedule never partitions the contraction dimension.

These observations map one-to-one onto the design: O1 $\rightarrow$ stream intermediates and batch candidates; O2 $\rightarrow$ hoist invariant projections; O3 $\rightarrow$ candidate batching and multicast; O4 $\rightarrow$ whole-contraction blocking.

Figure 1 places these components in a whole-decision budget. We distinguish the *reference expert step* (5.37 GMAC, including the invariant prefix K/V reprojection) from the *optimized repeated step* that hoists that 0.29 GMAC out of the loop (5.08 GMAC/step); either way the step is SwiGLU-dominated (3.54 GMAC). At $C=1$ the once-per-decision PyTorch prefill (203 GMAC) dwarfs the entire optimized K -step expert path (~ 51 GMAC at $K=10$: one hoisted

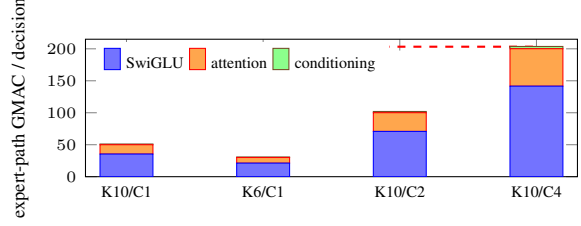


Figure 1. Whole-decision structural budget^[AN]. Stacked analytical MACs of the repeated expert path (SwiGLU / attention / conditioning) across operating points; the dashed line is the once-per-decision prefill, shared across candidates. The expert path is SwiGLU-dominated and only rivals the prefill once K and C grow. 0.29 GMAC of prefix reprojection is hoisted once per prepared context; the optimized repeated expert path is 5.08 GMAC per candidate-step. Non-MAC per-component metrics (simulator tiles, pipe events, storage) are tabulated separately in the artifact, not summed into a unified metric.

reprojection plus 10×5.08); only as candidates accumulate (expert path $\propto KC$) does the repeated portion approach the prefill—which is exactly why sharing one prefill across candidates (O3) is the highest-value reuse axis. Component metrics that are not commensurable (analytical MACs, simulator SwiGLU tiles, candidate-communication pipe events, immutable prefix storage) are reported side by side, not summed (t4.whole_decision.json).

4 TENSIXRT-VLA DESIGN

Figure 2 shows the system. The VLM prefill (tokenization, vision encoding, prefix K/V construction) remains in PyTorch; this boundary isolates the distinctive, repeated portion of the workload while using the reference implementation for the once-per-decision portion. Every floating-point operation after the prefix—suffix/action embedding, timestep conditioning, 16 expert layers, final norm, action projection, and the Euler update of Eq. 1—executes as TT-Lang kernels. Host code performs layout only (padding, slicing, head split/merge); a support matrix in the artifact enumerates every boundary, and no operation falls back silently.

Streaming SwiGLU. The expert MLP computes $\text{down}(\text{SiLU}(\text{gate}(x)) \odot \text{up}(x))$ with a $[50, 2048]$ intermediate. An unfused composition materializes gate, up, and product tensors to DRAM and re-reads them. Our kernel holds the padded $[64, 736]$ input resident in L1, streams 2-tile column blocks of the three weight matrices, forms $\text{SiLU}(xW_g) \odot xW_u$ as in-flight block temporaries, and accumulates the down-projection into a resident output buffer—the intermediate never reaches DRAM. Block sizes are chosen from the simulator’s L1 accounting (1,288 KiB of the 1,336 KiB budget at fp32); accumulation uses the

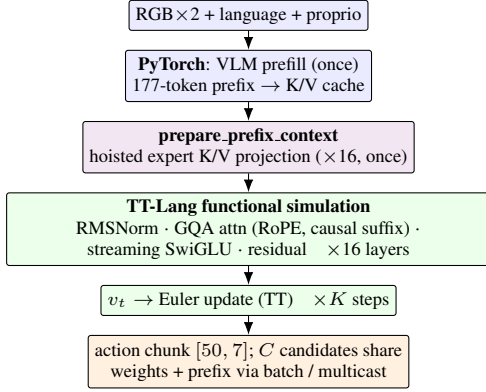


Figure 2. TensixRT-VLA. Blue: PyTorch (once per decision). Violet: invariant work hoisted out of the flow loop. Green: complete post-VLM path in TT-Lang functional simulation, repeated K times. No physical topology is depicted.

DFB ping-pong pattern required by the programming model.

GQA-aware attention. With 15 query heads over 5 K/V heads, we stack each group’s three query heads into one $[192, 64]$ operand so every K/V head is loaded once per group; K/V are never materialized per query head. Rotary embedding is expressed as a block-diagonal matrix product plus elementwise multiplies with precomputed tables, and the $1/\sqrt{d}$ scale folds into those constants. Masks implement the exact model semantics (invalid padded-language columns; causal suffix groups).

Prefix-K/V projection hoisting. On cross-attention layers the baseline re-projects the cached prefix K/V $([177, 320] \times [320, 320])$, twice per layer) inside every step and for every candidate: $C \cdot K \cdot 16$ projections per decision. TensixRT-VLA computes these once in `prepare_prefix_context` and exposes the projected, immutable result to all steps and candidates. This is a projection hoist, not a change to standard K/V caching.

Candidate parallelism. Two mappings are mathematically valid and complementary. *Batched- M* stacks candidates’ independent MLP rows $([C \cdot 50, 720])$, multiplying arithmetic intensity with no new weight traffic (attention keeps candidates as separate operands). *Multi-node partitioning* assigns candidates to core groups; shared immutable objects—weights and hoisted prefix—are then distributed by NoC multicast from a single reader instead of per-group DRAM reads, something the software cache abstraction (which copies per candidate) cannot express.

Numerical semantics. PyTorch fp32 is the reference throughout; validation uses real captured SmolVLA ten-

sors (the public `lerobot/smolvla.libero` checkpoint from the LeRobot library (Cadene et al., 2026): observations, prefix cache, per-step flow states). One deliberate deviation exists: the simulated attention softmax is numerically stable by construction. It subtracts the per-row maximum before the exponential (a row-max reduction) and is the default path. Max subtraction adds no simulator-observed DRAM transfers, DFB events, or additional buffer allocation in this implementation; it does add in-L1 reduction/compute, for which we make no timing claim^[SIM]. The stable implementation shows zero measured maximum absolute error against `torch.softmax` in the synthetic sweep through injected logit magnitude 200; the unstable variant (retained for A/B comparison) instead overflows fp32 at logit ≈ 88.7 , first failing at injected logit 90^[PT]. For context, tapping every attention logit over five observations $\times$ ten steps $\times$ sixteen layers, the peak magnitude is only 34.1, so both paths are correct on this workload; the stable default removes the dependence on that margin entirely.

5 METHODOLOGY

We organize evaluation around four questions. **RQ1** (fidelity): does the backend preserve SmolVLA behavior? Metric: max absolute error vs. PyTorch fp32 on real tensors, at layer, step, and free-running flow granularity^[PT]. **RQ2** (dataflow): how much structural movement do the transformations remove? Metric: simulator-observed tile transfers, DFB events^[SIM]; baseline: a reasonably blocked unfused composition. **RQ3** (reuse): how far do invariants amortize across K and C ? Metrics: projection call counts^[AN], per-candidate tile traffic^[SIM]. **RQ4** (robotics): how do flow-compute choices relate to closed-loop behavior? Metric: LIBERO (Liu et al., 2023a) task success under fixed and adaptive K ^[ROB], plus bit-exact counterfactual branching (environment state capture/restore validated to zero replay divergence, including a hidden gripper-integrator state we identified) and prefix-intervention rollouts (first m decisions shallow, remainder full-depth, on the matched grid). Statistics: episodes are paired on matched initial states; because they cluster within tasks, we report task-stratified paired bootstrap CIs and stratified sign-flip permutation tests alongside pooled Wilson intervals and exact McNemar tests. Workloads: real LIBERO observations; the `libero_10` suite stratified by measured difficulty for closed loop. All numbers derive from committed artifacts; category superscripts mark evidence type throughout.

Closed-loop protocol. All robotics rollouts execute the PyTorch reference SmolVLA policy (`lerobot/smolvla.libero`); the TT-Lang backend is validated against it offline (RQ1) and is *not* in the control loop, so RQ4 varies flow depth K in the reference model. Each policy call predicts the $H=50$ -

action chunk and all 50 actions execute before the next decision (full-chunk receding horizon); an episode ends on task success or the LIBERO per-task step limit (`env.max_episode_steps`: 520 for `libero_10` and 280 for `libero_spatial`, the suites evaluated here). For the counterfactual analysis we capture the environment state before a policy call, restore it bit-exactly, and evaluate depth branches $K \in \{2, 4, 6, 10\}$ from that state under matched noise; the “continuation” outcome then resumes the fixed baseline ($K=10$) policy with matched per-chunk noise for up to 150 further steps (“choose this depth now, then resume normal SmolVLA”). These continuations are therefore bounded-horizon: capped at 150 steps—well short of the 520-step `libero_10` limit—they can miss effects of the evaluated decision that only surface later in the episode. Branch states are sampled one per policy call across the evaluated episodes; the depth sensitivity set spans 147 such states pooled from the `libero_10` and `libero_spatial` tasks (spanning easy to hard). The learned per-decision scheduler is fit and evaluated on an episode-disjoint split (107 train / 40 val states, grouped by (suite, task, episode)), so no episode contributes to both training and evaluation.

Precision and memory accounting. Numerical-fidelity runs use fp32 unless explicitly stated; bf16 is used only for the stated expert weight-storage and operational-intensity estimates; simulator structural experiments use the precision documented for that experiment (fp32 for the SwiGLU traffic comparison, bf16 for the batched-candidate mapping). Two memory quantities are distinct and never conflated: the inter-layer hidden-state activation (144 KB, Table 1; the transient MLP intermediate is larger at 409.6 KB but is streamed, never fully materialized) and the *kernel/DFB working-set* L1 occupancy (e.g. 1,288 KiB for the fused SwiGLU), which counts the resident buffers a kernel allocates.

Evidence scope. Simulator-tagged results^[SIM] are *structural*: tile transfers, DFB synchronization events, and L1 capacity accounting of the authored dataflow under functional execution. They are not measurements of bandwidth, latency, throughput, energy, or on-silicon scheduling, and we draw no realized-performance conclusions from them anywhere in this paper (§8).

6 RESULTS

6.1 RQ1: Numerical fidelity

Single layers reproduce captured tensors to 1.4×10^{-6} (shared self-attention layer) and 2.6×10^{-6} (cross-attention layer) max absolute error. Divergence does not compound with depth (Fig. 3): across the 15 inter-layer taps of the 16-layer expert it spans only 4.3×10^{-6} to 7.6×10^{-6} .

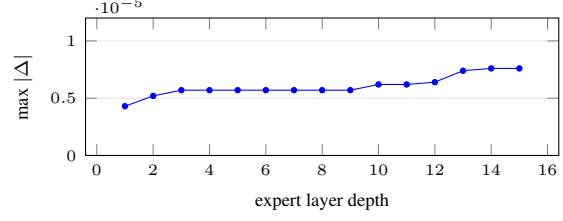


Figure 3. Backend divergence vs. PyTorch at the 15 inter-layer taps of the expert (denoise step 5, real tensors)^[PT]. Error stays flat; free-running $K=10$ integration ends at 8.3×10^{-7} final-action error.

Teacher-forced denoise steps against recorded flow states err at $\leq 6.2 \times 10^{-6}$, and complete free-running integration—every velocity, every Euler update, in the backend—matches PyTorch final actions to 8.3×10^{-7} ($K=10$) and 8.6×10^{-7} ($K=6$)^[PT]. Fidelity is a property of the workload, not of a curated case: a stress test of 240 free-running flows (ten observations $\times$ three rollout positions $\times K \in \{2, 4, 6, 10\} \times$ two seeds) has mean final-action error 1.6×10^{-6} and *worst-case* 3.4×10^{-5} , with Pearson correlation to the reference never below 0.9999993^[PT]. Errors are nonzero; we claim bounded fp32-level agreement, not bitwise equivalence (the hoisted path, by contrast, *is* bitwise-identical to the unhoisted path).

6.2 RQ2: Structural dataflow efficiency

The baseline applies the per-operator blocking and reuse used throughout our TT-Lang kernels: each weight is read once per matmul and the contraction dimension is fully blocked; it differs from the fused kernel only in cross-operator dataflow, so the reduction below combines two effects—eliminating the DRAM intermediate and improving input reuse (the fused kernel keeps the input resident across operators)—rather than reflecting a weak reference (appendix). On the real captured MLP case it moves 10,990 simulator-observed tiles per execution; the streaming kernel moves 4,508—a 59.0% reduction that *attains the derived padded tile-traffic lower bound under the stated tiling and DRAM-residency assumptions*, $T_{\min} = M_T D_T + 3 D_T I_T + M_T D_T = 46 + 3(1472) + 46 = 4,508$ for padded tiles $M_T=2$, $D_T=23$, $I_T=64$ (each mandatory operand moved once; the intermediate never reaches DRAM; derivation and assumptions in appendix), with DRAM writes down 89% and DFB events down 84%^[SIM]. Decomposing the unfused overage (6,482 tiles) attributes 3,584 to intermediate materialization and 2,898 to repeated input reads; tile padding, by contrast, adds only approximately 2.7% over the corresponding unpadded-element ideal. At $C=1$ the avoidable waste was materialization and re-reads, not the much-discussed padding (Fig. 4). Nor is the result specific to this case: across nine captured cases (expert layers $\{1, 8, 15\} \times$ three

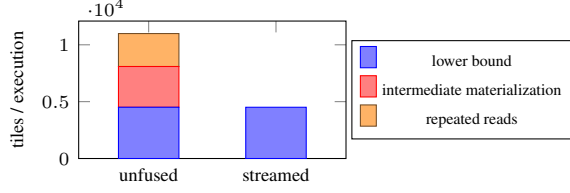


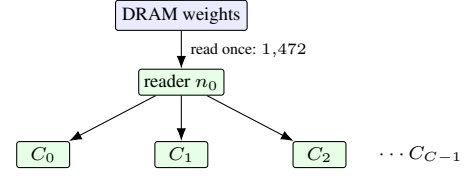
Figure 4. SwiGLU tile traffic, real captured tensors^[SIM]. The streamed kernel attains the derived padded tile-traffic lower bound (4,508 tiles) under the stated assumptions; the unfused coverage is materialization and re-reads, not padding.

observations) the unfused and streamed kernels move *identically* 10,990 and 4,508 tiles—traffic is shape-determined—with worst-case output error 3.6×10^{-6} ^[SIM]. These are authored-dataflow structural counts, not bandwidth or latency claims.

Two further checks harden the structural claims. First, the benefit is not tied to SmolVLA’s exact geometry: across a 64-point shape sweep ($M \in \{32, 50, 64, 96\} \times H \in \{512, 720, 768, 1024\} \times I \in \{1536, \dots, 4096\}$) the streamed kernel moves fewer tiles than the unfused one in *every* case (41–69% reduction), with the L1-capacity boundary appearing at $H=1024$ ^[SIM]; we do not claim these shapes are other models, only that intermediate elimination is not shape-fragile. Second, we cross-check the simulator with an *independently derived accounting model*: closed-form tile-traffic formulas obtained from the tensor geometry and the authored block schedule (not fitted to any simulator output; appendix). It agrees with the simulator on all 14 compared quantities (naive/streamed SwiGLU, batched weight tiles at every C , multicast/independent DRAM), max relative discrepancy 0^[AN]. We do not read this as two independent physical measurements—both describe the same deterministic authored schedule, so exact agreement is the expected outcome—but as evidence that the structural counts are transparently derivable from first principles rather than emergent artifacts of the simulator.

6.3 RQ3: Invariant and candidate reuse

Hoisting. Prefix-K/V projection calls fall from $C \cdot K \cdot 16$ to 16 per decision (e.g., $640 \rightarrow 16$ at $C=4$, $K=10$). At $C=4$, $K=10$ this removes 11.3 GMAC of repeated projection work *per decision* (summed over the four candidate trajectories and ten flow steps: $4 \cdot 10 \cdot 0.29$ GMAC of reprojection minus the single hoisted copy), for 3.6 MB of immutable storage; outputs are bitwise-identical to the unhoisted path on every observation tested (three fresh observations $\times$ two flow timesteps, $\max |\Delta|=0$)^[PT]. **Batching.** In the validated fused-MLP structural experiment, weight tiles per candidate are exactly $4416/C$ for $C \in \{1, 2, 4, 8\}$, total tiles per candidate fall $4,508 \rightarrow 627$, and DFB synchronization



multicast (B): weight DRAM = 1,472 (model: indep. of C ; valid. $C \in \{1, 2, 4, 8\}$); sends = 32; receives = $32(C-1)$. **independent (A):** weight DRAM = 1,472 C ; no pipes.

Figure 5. Weight-multicast topology and C -scaling^[SIM]. One reader fetches each weight tile once and forwards it over the NoC. The accounting model predicts weight DRAM independent of C (1,472), validated at the simulator operating points $C \in \{1, 2, 4, 8\}$; NoC delivery *grows* as $32(C-1)$. This is a structural trade (fewer DRAM reads, more NoC transfers); real silicon must determine whether it is performance-positive.

Table 2. Transformation summary. Metrics are per-transformation and not aggregable into a single number; “—” = not applicable. Tiles/events are simulator-observed^[SIM]; MACs analytical^[AN]. The batched $C=8$ row is idealized amortization (single-node L1 infeasible from $C=4$); the feasible large- C mapping is multi-node.

Configuration	$\max \Delta $	tiles	DFB ev.	notes
Unfused MLP baseline	1.3×10^{-6}	10,990	1,648	
+ streaming SwiGLU	1.3×10^{-6}	4,508	262	= padded min.
+ GQA group reuse	(in backend)	—	—	K/V $\times$ 1/group
+ prefix-K/V hoist	bitwise vs. base	—	—	$640 \rightarrow 16$ ($C4, K10$)
+ batched $C=8$	5.0×10^{-6}	627/cand.	262	L1 warn $C \geq 4$
+ 2-node multicast	bitwise A vs. B	1,472 wt	260	+1,472 pipe

events remain constant while the sequential baseline scales linearly^[SIM]. Two tiers must be distinguished: the single-node mapping is capacity-feasible only for $C \leq 2$ —from $C=4$ it exceeds the default L1 budget (simulator capacity warnings recorded), so the $C \in \{4, 8\}$ rows quantify *idealized amortization*, not a deployable single-node configuration; the feasible path for larger C is multi-node partitioning. **Multicast.** With two candidates on two nodes, independent loading reads 2,944 weight DRAM tiles; a single-reader NoC multicast reads 1,472, replacing the difference one-for-one with pipe transfers, with bitwise-identical outputs and unchanged DFB counts^[SIM]. Sweeping $C \in \{1, 2, 4, 8\}$, multicast weight DRAM reads stay fixed at 1,472 while independent loading grows exactly as $1,472 C$ (up to 11,776 at $C=8$); all outputs are bitwise-identical^[SIM]. The cost that grows is NoC delivery: pipe *send* events stay constant (32) but *receive* events scale as $32(C-1)$ (Fig. 5). So this trades DRAM re-reads for inter-node communication; over the tested range multicast weight-DRAM reads are C -invariant (so the saving over independent loading grows with C), but the real-silicon latency/energy consequence of the trade is unknown and is precisely what hardware validation must measure. Table 2 summarizes.

Table 3. Per-task closed-loop success by sustained depth^[ROB] ($n=25$ episodes per task per policy, matched initial states). Tasks ordered by `libero_10` id; 4 and 7 are the measured-hardest.

	task 0	task 1	task 4	task 7	task 9
fixed $K=2$	0.24	0.36	0.16	0.20	0.64
fixed $K=4$	0.32	0.32	0.24	0.28	0.68
fixed $K=6$	0.32	0.40	0.32	0.36	0.64
fixed $K=10$	0.28	0.40	0.32	0.32	0.76

Table 4. Closed-loop LIBERO success vs. sustained flow depth^[ROB] ($n=125$ episodes/policy, matched initial states; Wilson 95% CI). Paired exact McNemar: $K=10$ vs. $K=6$ $p=1.0$; $K=10$ vs. $K=2$ $p=0.0075$.

Policy	success	95% CI	expert evals/ep.
fixed $K=2$	0.320	[0.245, 0.406]	18.7
fixed $K=4$	0.368	[0.289, 0.455]	36.2
fixed $K=6$	0.408	[0.326, 0.496]	52.9
fixed $K=10$	0.416	[0.333, 0.504]	87.8
greedy adaptive (expl. $n=20$)	0.30	[0.15, 0.52]	28.2

6.4 RQ4: Closed-loop compute–quality

Table 4 reports fixed- K closed-loop success on five difficulty-stratified `libero_10` tasks with $n=125$ episodes per policy on *matched initial states*, enabling paired analysis. Because episodes cluster within tasks, we report, beyond pooled Wilson intervals, task-stratified paired bootstrap CIs on success deltas and stratified sign-flip permutation tests (which agree with the exact McNemar p -values). Sustained $K=2$ is significantly worse than $K=10$ ($\Delta=9.6$ pp, stratified 95% CI [3.2, 16.0]pp, $p=0.0075$) and than $K=6$ ($\Delta=8.8$ pp, CI [1.6, 16.0]pp, $p=0.027$). Sustained $K=6$ shows *no detected difference* from $K=10$ ($\Delta=0.8$ pp, CI [−4.8, +6.4]pp, $p=1.0$) at 60% of its expert compute (though, once the unchanged shared PyTorch prefill is included, this is only $\approx 8\%$ fewer whole-decision MACs at $C=1$: 254 $\rightarrow$ 234 GMAC); we note explicitly that this is absence of evidence, not equivalence—the CI does not exclude a 5pp advantage for $K=10$ at this sample size. Per-task rates (Table 3) show the $K=2$ deficit on every task and the hardest tasks (4, 7) benefiting most from depth. A second-suite spot-check on `libero_spatial` (five tasks, $n=50$ /policy, matched states) reproduces the *ordering*— $K=2$ lowest at 0.64, with $K=6$ and $K=10$ higher (0.74 and 0.70)—but the $K=2$ deficit is not significant on this easier, higher-baseline suite ($\Delta=6$ pp vs. $K=10$, CI [−4, 18]pp), consistent with depth mattering chiefly where tasks are hard. Every one of the five tasks favors deeper K (per-task $K=10 - K=2$ deltas 0.04–0.16), and the effect is heterogeneous: the hardest tasks (4, 7) gain the most (12–16 points), though the relation to difficulty is not monotone (the easiest task also gains 12 points), so the pooled effect is not driven by a single task^[ROB].

The interesting result is the contradiction in Fig. 6. Us-

ing bit-exact environment branching we evaluated the *same* states under $K \in \{2, 4, 6, 10\}$ with identical noise and a fixed continuation policy: depth changed the outcome at only 9 of 147 states, and an oracle matching fixed- $K=10$ branch success needs just 22% of its expert compute. Yet a scheduler trained on exactly these labels drifts to mean $K \approx 3$ and lands at 0.30 closed-loop—the always-cheap outcome its labels said it would avoid (Table 4). The counterfactual panel (Fig. 6a) pools branch states from `libero_10` and `libero_spatial` under 150-step labels, whereas the sustained and intervention panels (b–d) are full-episode rollouts on the five `libero_10` tasks. Because task populations and evaluation horizons differ, the cross-panel contrast is descriptive; the within-suite interventions and exploratory scheduler results provide evidence consistent with the proposed mechanism.

Mechanism: displacement by early shallow decisions.

We probe the mechanism with *prefix-intervention* rollouts: the first m decisions run at $K=2$, every later decision at $K=10$, on the same matched episode grid ($n=100$ pairs). The $m=0$ and $m=\infty$ arms reproduce the fixed- $K=10$ and fixed- $K=2$ episodes *exactly* (100/100 outcome matches each), confirming the harness is deterministic and the arms differ only in the depth schedule. The measured drop appears by $m=2$ and stays flat thereafter: success falls 0.40 $\rightarrow$ 0.33 at $m=2$ (paired $\Delta=7$ pp, stratified 95% CI [0, 14]pp—the CI touches zero, so we read this as mechanism evidence, corroborated by the significant sustained- $K=2$ result, not a standalone significance claim) and does not fall further at $m \in \{4, 8, \infty\}$ (all 0.33; Fig. 6c). Restoring full depth after a shallow start does not visibly recover: the $m=4$ arm runs every decision after the fourth at full depth—61 vs. the pure-shallow arm’s 19 flow steps/episode^[ROB], a $\sim 3\times$ compute increase—yet lands at the same 0.33. The paired state trajectories show why: the same-episode proprioceptive distance between the pure- $K=2$ and pure- $K=10$ arms grows monotonically with decision index (Fig. 6d). This is consistent with early shallow chunks displacing the rollout onto states from which later full-depth decisions do not visibly return. These intervention rollouts are *not* truncated: each runs to task success or the 520-step `libero_10` episode limit (hence the $m=4$ arm’s 61 flow steps/episode); the 150-step cap applies only to the counterfactual branch labels, not here. The displacement is weakly predictive of individual failure—episodes whose shallow arm diverges more in the first two decisions are somewhat more likely to fail (point-biserial $r=0.11$; mean early divergence 0.059 before failures vs. 0.046 before successes)—and the deficit varies across tasks^[ROB]; we report these as correlational, not causal. Single-decision branch labels, which probe states reached under a full-depth history, cannot see this displacement—and, being capped at 150 continuation steps, also miss consequences that only resolve later in the episode—which is

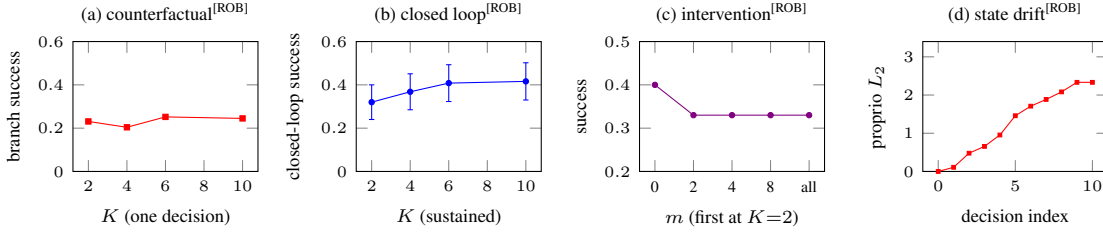


Figure 6. The K paradox: *locally recoverable is not globally free^[ROB]*. (a) Same-state single-decision counterfactual branches (bit-exact restore, fixed continuation; 147 states pooled from `libero_10` and `libero_spatial`, 150-step labels) are nearly flat in K —depth changes the outcome at 6.1% of states. (b) Sustained closed-loop success rises with K (Wilson 95% CIs; $n=125$ /policy; $K=2$ vs. $K=10$ paired $p=0.0075$; $K=6$ vs. $K=10$ no detected difference, CI does not exclude a $K=10$ advantage). (c) Prefix intervention: running the first m decisions at $K=2$ then $K=10$; the deficit appears by $m=2$ and does not recover (the $m \geq 2$ delta CI touches zero—mechanism evidence corroborated by (b), not standalone significance). (d) Same-episode proprioceptive distance between the pure- $K=2$ and pure- $K=10$ arms grows monotonically with decision index. Panels (b–d) are full-episode `libero_10` rollouts; (a) differs in task population and horizon, so the (a)-vs-(b) contrast is descriptive.

how a label-trained scheduler can be licensed to choose cheap K early. We conclude, for this setup: *single-decision counterfactual labels are biased toward shallow choices in this evaluated closed-loop setting*—for the same-state counterfactual labels evaluated here, locally sufficient shallow decisions can still induce a sustained closed-loop deficit. This does not rule out adaptive computation, or greedy supervision in general; the greedy label-trained scheduler we tested (exploratory, $n=20$) failed to preserve closed-loop performance, which motivates sequential credit assignment over greedy per-decision supervision on these labels.

7 RELATED WORK

Efficient VLA inference. Flow-matching VLAs descend from π_0 (Black et al., 2025a) and are made affordable by SmolVLA (Shukor et al., 2025); a parallel line shrinks or restructures the model itself: compact backbones (Wen et al., 2025), 1-bit weights (Wang et al., 2025a), parallel decoding with continuous action heads (Kim et al., 2025), compressed action tokenizations (Pertsch et al., 2025), and speculative decoding (Wang et al., 2025b). Training-free serving optimizations reuse temporally static visual tokens (Xu et al., 2025), prune layers and tokens (Yang et al., 2025), or route between models of different sizes (Li et al., 2025). Closest in spirit to our RQ4, DeeR-VLA (Yue et al., 2024) allocates compute per decision by early exit, following the classic anytime-inference recipe (Teerapittayanon et al., 2016; Huang et al., 2018). Among the works we survey these operate at the model or serving-software level on general-purpose hardware rather than mapping the policy onto an explicit-dataflow accelerator. These methods demonstrate several forms of adaptive per-decision computation; our result is narrower: for the same-state counterfactual labels evaluated here, locally sufficient shallow decisions can still induce a sustained closed-loop deficit (§6.4).

Accelerating diffusion and flow policies. Diffusion Policy (Chi et al., 2023) established iterative action sampling for visuomotor control, with flow matching (Lipman et al., 2023; Liu et al., 2023b) as the few-step formulation VLAs now adopt. Acceleration work reduces sampling steps by retraining—consistency distillation (Prasad et al., 2024; Gao et al., 2024), one-step distillation (Wang et al., 2024), frequency consistency (Su et al., 2025)—or reschedules a fixed model, streaming partially denoised chunks (Høeg et al., 2024) or overlapping generation with execution (Black et al., 2025b). Recent systems work has moved closer to the inference-runtime boundary: `vla.cpp` (Nguyen et al., 2026) implements native flow/diffusion VLA serving with cached prefixes across heterogeneous hardware, while Realtime-VLA FLASH (Niu et al., 2026) uses speculative action generation and flow-based verification; ElegantVLA (Li et al., 2026) and learned adaptive token-caching methods (Wei et al., 2026) further allocate computation by temporal or task-dependent signals. TensixRT-VLA is complementary: we hold the trained model and its K -step semantics fixed and study the explicit dataflow mapping and structural movement of the repeated action expert on a tiled many-core target; and where step-reduction methods certify quality per decision, our results caution that per-decision equivalence does not guarantee sustained closed-loop equivalence.

Test-time action search. Sampling multiple action candidates and selecting among them improves generalist policies: value-guided re-ranking (Nakamoto et al., 2024), human-in-the-loop steering (Wang et al., 2025c), verifier-based selection (Dai et al., 2025), and sampling-scaling laws for VLAs (Kwok et al., 2025). These works presuppose that candidate generation is worth its cost; we quantify its *structural* cost basis on a tiled target—in the fused-MLP mapping (tested $C \in \{1, 2, 4, 8\}$) batched candidates consume weight tiles at exactly $1/C$ and multicast makes weight

DRAM traffic C -invariant (§6.3)—while our label-trained scheduler failure (§6.4) flags that the labels used to train such selectors need sequential, not single-decision, validation.

Dataflow accelerators and kernel mapping. Tensix-class devices (Vasiljevic et al., 2021) expose a compute-on-packets NoC fabric, one point in a spatial-architecture space that includes deterministic streaming (Abts et al., 2020) and wafer-scale broadcast fabrics (Lie, 2024). Tile-level programming and fusion are established at the DSL/compiler level (Tillet et al., 2019; Chen et al., 2018) and in IO-aware kernels whose traffic analysis we mirror (Dao et al., 2022; Williams et al., 2009); we author kernels directly in TT-Lang (Tenstorrent Inc., 2026a) over the TT-Metalium stack (Tenstorrent Inc., 2026b). Closest to our systems contribution, Wu et al. (2026) fuse RMSNorm into matmuls and use NoC multicast for LLM decoding on Wormhole silicon, and Taylor et al. (2026) study numerical-kernel mappings on the same device class. TensixRT-VLA differs in scope and object: we map a *complete post-VLM* flow-matching action-generation path, drive fusion to the measured tile-traffic lower bound, multicast weights across *candidate trajectories* rather than decode streams, and—among the systems we survey—connect the mapping to closed-loop robot behavior. We do not claim NoC multicast as a new architectural primitive—Tenstorrent documents shared-tile NoC distribution—our contribution is the candidate-sharing mapping and its structural accounting (DRAM-read vs. NoC-delivery, with bitwise-equivalent candidate outputs) for iterative VLA inference. A structured comparison matrix accompanies the artifact (`related_work_matrix.csv`).

8 LIMITATIONS AND DISCUSSION

Hardware gap. Physical Tenstorrent silicon was not available for this draft. TT-Lang functional simulation validates program and dataflow correctness, numerical output, structural tile transfers, and DFB/L1 accounting of the authored dataflow. It does not validate latency, throughput, bandwidth, energy, or silicon scheduling effects. Same-source compilation and on-device measurement are the most important missing experiments.

Generality beyond SmolVLA. Our results are a single-VLA case study, but the exploited structure is not idiosyncratic to SmolVLA. The flow-matching VLA π_0 (Black et al., 2025a) shares the same three invariants by construction: it integrates a flow-matching *action expert* (300M parameters, separate from a PaliGemma backbone) for a default of 10 Euler steps over an $H=50$ action chunk, and its published inference explicitly “cach[es] the attention keys and values for the prefix... and only recomput[es] the suffix... for each integration step”—precisely the invariant-prefix / iterated-weight-dominated-expert structure that O1–

Table 5. Generality of each insight across three iterative policies. **Only SmolVLA is implemented;** for SmolVLA a $\checkmark$ on a mapping row (SwiGLU, hoist, batching, multicast) denotes an implemented+measured kernel, and on a workload row a measured property. For π_0 /Diffusion Policy every cell is a *characterization*, never an implemented kernel: $\checkmark$ =verified workload property, $\circ$ =plausible opportunity (not built in), var.=variant-dependent, hyp.=hypothesis, $\times$ =absent. Class: **S**=SmolVLA-specific, **F**=flow/transformer-policy level, **I**=broadly iterative-policy level.

Insight	SmolVLA	π_0	Diff. Pol.	Class
Iterative action generation	$\checkmark$	$\checkmark$	$\checkmark$	I
Invariant obs./prefix context	$\checkmark$	$\checkmark$	$\checkmark$	I
Repeated weight execution	$\checkmark$	$\checkmark$	$\checkmark$	I
Candidate-sharing opportunity	$\checkmark$	$\circ$	$\circ$	I
Transformer action expert	$\checkmark$	$\checkmark$	var.	F
SwiGLU (gated-MLP) elimination	$\checkmark$	gated [†]	$\times$	S/F
Prefix-projection hoist	$\checkmark$	$\circ$ [‡]	var.	F
Candidate batching	$\checkmark$	$\circ$	$\circ$	I
Weight-multicast opportunity	$\checkmark$	$\circ$	$\circ$	I
Closed-loop depth caution	meas.	hyp.	hyp.	I

[†] π_0 ’s Gemma-based expert uses a gated MLP (plausible target; unverified for the SiLU-gated form). [‡] π_0 caches prefix K/V; whether a re-projection remains to hoist is implementation-dependent.

O2 target, with candidate trajectories again sharing weights and prefix/context while differing in their initial noise (O3). The concrete mappings would differ in named ways: π_0 ’s action tokens use a *bidirectional* mask where SmolVLA’s suffix is causal, so the attention kernel changes. Candidate batching follows from the shared-weight structure at the workload level; streaming over π_0 ’s gated MLP and any prefix-projection hoist are plausible but implementation-dependent and unvalidated here. We do not implement π_0 ; we claim only that the workload characterization, not the specific kernels, is likely to transfer. A verified reading of a structurally different policy, Diffusion Policy (Chi et al., 2023), bounds this claim precisely. We separate three kinds of claim. (i) *Transferable workload invariants*: it too runs a separate weight-dominated denoiser K times against an observation feature computed once and held fixed across those steps (O1, O2), inside a receding-horizon loop. (ii) *Transferable mapping opportunities*: candidate batching and weight multicast follow from those invariants and apply directly, whereas the streaming-SwiGLU kernel is SmolVLA-specific (its default U-Net has no SwiGLU) and the projection hoist applies only to its transformer variant (its default has no attention prefix). (iii) *A closed-loop caution, not a transformation*: the K-paradox is a hypothesis about iterative-refinement policies in a control loop, which Diffusion Policy’s receding-horizon structure makes plausible but which we have not tested on it. The invariants recur; the most model-specific kernel does not. Table 5 summarizes; π_0 and Diffusion Policy are characterized from their papers and source, *not implemented*, so no entry implies validation on them.

Further limitations: the VLM prefill remains in PyTorch; closed-loop samples remain moderate ($n=125$ /policy on five `libero_10` tasks plus a `libero_spatial` spot-check, and the learned-scheduler run is exploratory $n=20$); and the greedy adaptive- K negative result does not preclude sequential adaptive computation—it motivates it. Functional simulator execution does not imply compatibility with all versions of the TT-Lang compiler; representative compile-only validation was not performed here, as the compiler toolchain was unavailable in our environment. (The earlier bounded-logit softmax assumption is resolved: the stable max-subtracted path is now the default and adds no simulator-observed structural cost, §4.)

Three design principles emerge that do not depend on SmolVLA or Tensix specifics. (P1) *Hoist invariant computation before optimizing repeated computation*: the cheapest flow step is work moved outside the loop, and iterative policies make loop-invariance unusually common. (P2) *When weights cannot stay resident, batch independent work against one weight pass*: candidate trajectories are a natural, algorithm-provided batching axis whose marginal cost we measure at exactly $1/C$ in weight traffic in the fused-MLP mapping (tested $C \in \{1, 2, 4, 8\}$). (P3) *Evaluate compute reductions under the closed-loop state distribution*: per-decision quality deltas mispredict rollout outcomes because the policy’s future inputs depend on its past compute.

9 CONCLUSION

TensixRT-VLA demonstrates that flow-matching VLA inference carries exploitable structure—invariant context, repeated weight-dominated expert execution, and candidate-level sharing—that a tiled dataflow target can express directly: a complete post-VLM backend reproducing PyTorch integration to 8.3×10^{-7} , streaming fusion at the derived tile-traffic lower bound under the stated assumptions, projection hoisting to a one-time per-context cost, and measured candidate multicast. The closed-loop analysis contributes a caution for the growing efficient-inference literature: compute savings certified per decision can evaporate, or invert, under the state distribution the policy itself induces. Validating these structural results on physical silicon is the immediate next step.

REFERENCES

- Abts, D., Ross, J., Sparling, J., Wong-VanHaren, M., Baker, M., Hawkins, T., Bell, A., Thompson, J., Kahsai, T., Kimmell, G., Hwang, J., Leslie-Hurd, R., Bye, M., Creswick, E., Boyd, M., Venigalla, M., Laforge, E., Purdy, J., Kamath, P., Maheshwari, D., Beidler, M., Rosseel, G., Ahmad, O., Gagarin, G., Czekalski, R., Rane, A., Parmar, S., Werner, J., Sproch, J., Macías, A., and Kurtz, B. Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 145–158, 2020. doi:10.1109/ISCA45697.2020.00023.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., Tanner, J., Vuong, Q., Walling, A., Wang, H., and Zhilinsky, U. π_0 : A Vision-Language-Action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025a. arXiv:2410.24164.
- Black, K., Galliker, M. Y., and Levine, S. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b. arXiv:2506.07339.
- Cadene, R., Alibert, S., Capuano, F., Aractingi, M., Zouitine, A., Kooijmans, P., Choghari, J., Russi, M., Pascal, C., Palma, S., Shukor, M., Moss, J., Soare, A., Aubakirova, D., Lhoest, Q., Gallouédec, Q., and Wolf, T. LeRobot: An open-source library for end-to-end robot learning. In *International Conference on Learning Representations (ICLR)*, 2026. Library commit 2774d9b used in this work.
- Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 578–594, 2018. arXiv:1802.04799.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion Policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2023. arXiv:2303.04137; originally RSS 2023.
- Dai, M., Liu, L., Bai, Y., Liu, Y., Wang, Z., Su, R., Chen, C., Lin, L., and Wu, X. RoVer: Robot reward model as test-time verifier for Vision-Language-Action model. *arXiv preprint arXiv:2510.10975*, 2025.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135.
- Gao, Z., Lu, G., Chen, T., Dai, W., Wang, Z., Shang, C., Ding, W., and Tang, Y. ManiCM: Real-time 3d diffusion policy via consistency model for robotic manipulation. *arXiv preprint arXiv:2406.01586*, 2024.

- Høeg, S. H., Du, Y., and Egeland, O. Streaming Diffusion Policy: Fast policy synthesis with variable noise diffusion models. *arXiv preprint arXiv:2406.04806*, 2024.
- Huang, G., Chen, D., Li, T., Wu, F., van der Maaten, L., and Weinberger, K. Q. Multi-Scale Dense Networks for resource efficient image classification. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1703.09844.
- Kim, M. J., Finn, C., and Liang, P. Fine-tuning Vision-Language-Action models: Optimizing speed and success. In *Robotics: Science and Systems (RSS)*, 2025. arXiv:2502.19645.
- Kwok, J., Agia, C., Sinha, R., Foutter, M., Li, S., Stoica, I., Mirhoseini, A., and Pavone, M. RoboMonkey: Scaling test-time sampling and verification for Vision-Language-Action models. In *Conference on Robot Learning (CoRL)*, 2025. arXiv:2506.17811.
- Li, Y., Meng, Y., Sun, Z., Ji, K., Tang, C., Fan, J., Ma, X., Xia, S., Wang, Z., and Zhu, W. SP-VLA: A joint model scheduling and token pruning approach for VLA model acceleration. *arXiv preprint arXiv:2506.12723*, 2025.
- Li, Y., Liu, H., Ji, K., Meng, Y., Fan, J., Wang, Y., Qin, S., Wu, C., Xia, S.-T., and Wang, Z. ElegantVLA: Learning when to think for efficient vision-language-action models. *arXiv preprint arXiv:2605.29438*, 2026.
- Lie, S. Inside the Cerebras wafer-scale cluster. *IEEE Micro*, 44(3):49–57, 2024. doi:10.1109/MM.2024.3386628.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.02747.
- Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023a. arXiv:2306.03310.
- Liu, X., Gong, C., and Liu, Q. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023b. arXiv:2209.03003.
- Nakamoto, M., Mees, O., Kumar, A., and Levine, S. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning (CoRL)*, 2024. arXiv:2410.13816.
- Nguyen, K. D., Ho, H. T., Nguyen, C. T., Duong, T. Q., Le, L. D., Nguyen, D. M. H., Ngo, V. A., and Le, A. T. vla.cpp: A unified inference runtime for vision-language-action models. *arXiv preprint arXiv:2606.08094*, 2026.
- Niu, J., Gu, K., Zhao, Y., Liang, S., Wang, T., Hu, X., Wang, Y., and Li, H. Realtime-VLA FLASH: Speculative inference framework for diffusion-based vlans. *arXiv preprint arXiv:2605.13778*, 2026.
- Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., and Levine, S. FAST: Efficient action tokenization for Vision-Language-Action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Prasad, A., Lin, K., Wu, J., Zhou, L., and Bohg, J. Consistency Policy: Accelerated visuomotor policies via consistency distillation. *arXiv preprint arXiv:2405.07503*, 2024.
- Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., Alibert, S., Cord, M., Wolf, T., and Cadene, R. SmolVLA: A Vision-Language-Action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- Su, Y., Liu, N., Chen, D., Zhao, Z., Wu, K., Li, M., Xu, Z., Che, Z., and Tang, J. FreqPolicy: Efficient flow-based visuomotor policy via frequency consistency. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.08822.
- Taylor, M., Pearson, C., Berger-Vergiat, L., Long, G., and Ciesko, J. Numerical kernels on a spatial accelerator: A study of Tenstorrent Wormhole. In *IEEE High Performance Extreme Computing Conference (HPEC)*, 2026. arXiv:2603.23343.
- Teerapittayanon, S., McDanel, B., and Kung, H. T. BranchyNet: Fast inference via early exiting from deep neural networks. In *International Conference on Pattern Recognition (ICPR)*, 2016. arXiv:1709.01686.
- Tenstorrent Inc. tt-lang: A python DSL for tenstorrent kernel authoring. <https://github.com/tenstorrent/tt-lang>, 2026a. tt-lang-sim 1.1.6; repo main @317250e surveyed.
- Tenstorrent Inc. TT-Metalium. <https://github.com/tenstorrent/tt-metal>, 2026b.
- Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pp. 10–19, 2019. doi:10.1145/3315508.3329973.
- Vasiljevic, J., Bajić, L., Capalija, D., Sokorac, S., Ignjatovic, D., Bajic, L., Trajkovic, M., Hamer, I., Matosevic, I., Cejkov, A., Aydonat, U., Zhou, T., Gilani, S., Paiva, A., Chu, J., Maksimovic, D., Chin, S., Moudallal, Z., Rakhmati, A., Nijjar, S., Bhullar, A., Drazic, B., Lee,

- C., Sun, J., Kwong, K.-M., Connolly, J., Dooley, M., Farooq, H., Chen, J., Walker, M. J. P., Dabiri, K., Mabee, K., Lal, R. S., Rajatheva, N., Retnamma, R., Karodi, S., Rosen, D., Muñoz, E., Lewycky, A., Knezevic, A., Kim, R., Rui, A., Drouillard, A., and Thompson, D. Compute substrate for software 2.0. *IEEE Micro*, 41(2):50–55, 2021. doi:10.1109/MM.2021.3061912.
- Wang, H., Xiong, C., Wang, R., and Chen, X. BitVLA: 1-bit Vision-Language-Action models for robotics manipulation. *arXiv preprint arXiv:2506.07530*, 2025a.
- Wang, S., Yu, R., Yuan, Z., Yu, C., Gao, F., Wang, Y., and Wong, D. F. Spec-VLA: Speculative decoding for Vision-Language-Action models with relaxed acceptance. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025b. arXiv:2507.22424.
- Wang, Y., Wang, L., Du, Y., Sundaralingam, B., Yang, X., Chao, Y.-W., Perez-D’Arpino, C., Fox, D., and Shah, J. Inference-time policy steering through human interactions. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025c. arXiv:2411.16627.
- Wang, Z., Li, Z., Mandlekar, A., Xu, Z., Fan, J., Narang, Y., Fan, L., Zhu, Y., Balaji, Y., Zhou, M., Liu, M.-Y., and Zeng, Y. One-Step Diffusion Policy: Fast visuomotor policies via diffusion distillation. *arXiv preprint arXiv:2410.21257*, 2024.
- Wei, Y., Fan, J., Guo, J., Zhen, R., Shao, R., Su, X., Xie, Z., Yao, H., and Yang, S. Learning to accelerate vision-language-action models through adaptive visual token caching. *arXiv preprint arXiv:2602.00686*, 2026.
- Wen, J., Zhu, Y., Li, J., Zhu, M., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., Peng, Y., Feng, F., and Tang, J. TinyVLA: Towards fast, data-efficient Vision-Language-Action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025. arXiv:2409.12514.
- Williams, S., Waterman, A., and Patterson, D. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- Wu, Q., Li, K., Wang, W., Yu, J., Zhang, R., and Liu, L. Operator fusion for LLM inference on the Tensix architecture. *arXiv preprint arXiv:2606.09879*, 2026.
- Xu, S., Wang, Y., Xia, C., Zhu, D., Huang, T., and Xu, C. VLA-Cache: Efficient Vision-Language-Action manipulation via adaptive token caching. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.02175.
- Yang, Y., Wang, Y., Wen, Z., Luo, Z., Zou, C., Zhang, Z., Wen, C., and Zhang, L. EfficientVLA: Training-free acceleration and compression for Vision-Language-Action models. *arXiv preprint arXiv:2506.10100*, 2025.
- Yue, Y., Wang, Y., Kang, B., Han, Y., Wang, S., Song, S., Feng, J., and Huang, G. DeeR-VLA: Dynamic inference of multimodal large language models for efficient robot execution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2411.02359.

A STRUCTURAL TILE-TRAFFIC LOWER BOUND

Tiles are 32×32 . For the canonical SmolVLA MLP ($X[50, 720]$, $W_g, W_u[720, 2048]$, $W_d[2048, 720]$) the padded tile counts are $M_T = \lceil 50/32 \rceil = 2$, $D_T = \lceil 720/32 \rceil = 23$, $I_T = \lceil 2048/32 \rceil = 64$ (padded input $[64, 736]$).

Operand / movement	Tiles
padded input X read, 2×23	46
gate weights W_g , 23×64	1,472
up weights W_u , 23×64	1,472
down weights W_d , 64×23	1,472
output Y write, 2×23	46
Total	4,508

Assumptions. (i) one kernel invocation; (ii) the current padded layout and stated precision (fp32); (iii) mandatory input operands (X, W_g, W_u, W_d) begin in DRAM; (iv) the output Y must be emitted to DRAM; (v) no persistent weight residency across invocations; (vi) the SiLU-gate intermediate remains on-chip. Under these assumptions, every mandatory operand must move at least once, so $T_{\min} = M_T D_T + 3 D_T I_T + M_T D_T = 46 + 3(1472) + 46 = 4,508$ tiles. This is a bound for the authored schedule under (i)–(vi); it is *not* an unconditional hardware lower bound. Batching amortizes the $3 D_T I_T$ weight term across candidates; persistent weight residency (shown infeasible on the assumed single-device budget) would remove it from a per-step bound.

B UNFUSED BASELINE SCHEDULE

The baseline is three tiled matmuls (gate, up, down) plus an elementwise SiLU-gate combine. Each matmul reads every weight exactly once and blocks the whole contraction dimension; intermediates G, U, H ($[50, 2048]$ each) are written to DRAM and read back, and X is re-read once per output tile-column in gate/up. The fused kernel differs by streaming across operators—keeping the intermediate off DRAM and X resident—so the measured reduction combines eliminated intermediate materialization and improved input reuse. It is not a strawman: the overage over the bound is $10,990 - 4,508 = 6,482 = 3,584$ intermediate materialization + $2,898$ input re-reads; tile padding adds only $\approx 2.7\%$ over the unpadded-element ideal.

C INDEPENDENTLY DERIVED ACCOUNTING MODEL

The tile-traffic formulas are derived from tensor geometry and the authored block schedule (granularities $N_GRAN=2$, $E_GRAN=DK_GRAN=8$, $DN_GRAN=1$), *not* fitted to simulator traces:

$$\begin{aligned} \text{naive} &= \underbrace{2 \left[\frac{I_T}{2} M_T D_T + D_T I_T + M_T I_T \right]}_{\text{gate+up}} + \underbrace{3 M_T I_T}_{\text{SiLU-mul}} + \underbrace{(D_T M_T I_T + I_T D_T + M_T D_T)}_{\text{down}} = 10,990, \\ \text{streamed} &= M_T D_T + 3 D_T I_T + M_T D_T = 4,508. \end{aligned}$$

Batched weight tiles per candidate $= 3 D_T I_T / C = 4416 / C$; multicast weight DRAM $= D_T I_T = 1,472$ independent of C ; independent loading $= 1,472 C$. These match the simulator exactly on all 14 compared quantities. Because both the model and the simulator describe the same deterministic authored schedule, exact agreement is expected; this is an independently derived accounting cross-check, *not* an independent physical validation.

D REPRODUCIBILITY AND PROVENANCE

Software. tt-lang-sim 1.1.6 (PyPI, hardware-free functional simulator); Python 3.12.10; PyTorch 2.14.0 (simulator env) / 2.11.0 (LeRobot env); transformers 5.5.4; LeRobot 0.6.2 at commit 2774d9b. **Model / data.** public lerobot/smolvla-libero checkpoint; LIBERO benchmark via the lerobot/libero dataset. **Host.** macOS arm64, CPU fp32 (canonical reference); no physical Tenstorrent silicon. **Seeds.** closed-loop evaluation uses eval seed 500 with matched episode_index across K (paired); stratified bootstrap/permutation tests use seed 20260912. **Discordant pairs (paired exact McNemar, $n=125$).** $K=10$ vs. $K=6$: (7, 6), $p=1.0$; $K=10$ vs. $K=2$: (15, 3), $p=0.0075$; $K=6$

vs. $K=2$: (16, 5), $p=0.027$. **Code and artifacts.** Public release: <https://github.com/lc99/tensixrt-vla> (release v1.0-preprint). Canonical numbers in `results/master_results.yaml`; per-experiment artifacts under `artifacts/` (e.g. `ttl原因/swiglu-comparison.json`, `ttl原因/s5-analytical-vs-sim.json`, `phase9/el-paired-stats.json`); generation scripts under `scripts/` and make targets (`reproduce-swiglu`, `reproduce-multicast`, `reproduce-analytical`). A drift test suite asserts the manuscript numbers against the canonical file.

E EVIDENCE TAG LEGEND

[PT] PyTorch / reference numerical measurement (fp32 golden). **[SIM]** TT-Lang functional-simulator structural measurement (tile transfers, DFB events, L1 accounting of the authored dataflow; not timing). **[AN]** analytical calculation / model. **[ROB]** closed-loop robotics experiment (LIBERO). No essential claim depends on appendix-only evidence.